Smart Home Model

Mario Belmonte, Cindy Chen

Department of Electrical and Computer Engineering, Carnegie Mellon University

***Abstract*: This report presents a web-first smart building model that integrates RFID access control, PID-based HVAC temperature regulation, and adaptive lighting control using ESP32 microcontrollers. All sensing and actuation are centrally managed through a web dashboard, achieving <500 ms access response and <1 s environmental updates. A PID-controlled fan system maintains temperature within ±1 °C of a setpoint, while ambient-light-driven dimming automatically adjusts artificial lighting for energy efficiency. Compared to passive monitoring systems, this platform demonstrates real-time, closed-loop environmental control with secure centralized management, validating the feasibility of responsive, fully web-dependent building automation.**

***Index Terms*: Access control, building automation, daylight harvesting, ESP32, HVAC control, Internet of Things, PID control, RFID, smart lighting, web-based control**

I. Introduction

MODERN buildings increasingly rely on automated systems to manage access control, lighting, and climate to improve security, energy efficiency, and occupant comfort. Traditionally, these subsystems operate as independent units with local controllers and limited integration, making centralized management difficult and reducing overall system visibility. Building managers and security personnel benefit from unified platforms that provide real-time monitoring, historical analytics, and remote control of critical infrastructure. Our project addresses this need by developing a physical model building that integrates access control, HVAC temperature regulation, and lighting automation into a single web-based control system. As described in our design review presentation, the intended users are building managers and security personnel who require centralized RFID door access, lighting control, and environmental monitoring through a unified dashboard interface.

Existing commercial "smart home" and building automation products, such as standalone smart locks and thermostats, provide basic remote functionality but typically rely on proprietary ecosystems and distributed local control logic. These systems limit flexibility, make integration across vendors difficult, and reduce transparency into system operation. Furthermore, many systems prioritize convenience over performance guarantees, providing limited quantitative assurance of response latency, system availability, or data integrity. In contrast, our project explores a fully web-dependent architecture in which all sensing, decision-making, and actuation are coordinated through a centralized backend. This approach allows precise policy enforcement, complete system observability, and consistent integration across subsystems.

The goal of this project is to design and implement a smart building model capable of secure RFID-based access control, closed-loop HVAC temperature regulation using PID-controlled fans, and adaptive lighting through daylight harvesting and PWM dimming. The system provides real-time environmental monitoring and user control through a web dashboard, achieving performance targets such as door unlock latency under 500 ms, environmental and lighting updates within 1 second, and rapid lighting response to ambient changes. By demonstrating reliable, low-latency operation using centralized web infrastructure, this project evaluates the feasibility, performance, and limitations of fully integrated, web-first building automation systems.

II. Use-Case Requirements

The primary use case for this system is centralized building management through a web-based platform that allows building managers and security personnel to monitor and control access, lighting, and temperature in real time. The system must provide reliable, low-latency responses to physical and environmental events while maintaining secure operation and continuous availability. To meet this use case, several quantitative performance requirements must be satisfied. These requirements were identified with consideration for public health, safety, and welfare, as well as environmental and economic factors. Safety and welfare drive the access control and availability targets: reliable, low-latency door response and 100% logging reduce security risk and support occupant safety; 99.9% availability and secure authentication protect building operations and data. Occupant welfare motivates the HVAC and lighting specifications, temperature within a comfort band and consistent illumination support a usable, comfortable environment. Environmental and economic factors motivate daylight harvesting and efficient control: automatic lighting adjustment in response to ambient light reduces energy use and cost while meeting illumination needs.

Access Control Requirements:

Building security personnel must be able to grant or deny entry quickly and reliably using RFID credentials. Slow response times can cause user frustration and create security vulnerabilities. Therefore, the system must detect RFID credentials within 50 ms of presentation and actuate the door lock within 500 ms of the initial swipe. Additionally, access permissions must be enforceable in real time, requiring revoked credentials to be rejected within 100 ms of a policy change. The system must also log 100% of access attempts with timestamps to ensure a complete audit trail for security review.

HVAC Temperature Regulation Requirements:

Facility managers require the ability to maintain indoor temperature within a specified comfort range and monitor environmental conditions over time. The system must measure ambient temperature at a minimum sampling rate of 1 Hz to ensure sufficient responsiveness for control. Using a closed-loop PID control system, the HVAC fan must regulate temperature to within ±0.5 °C of a user-defined setpoint under steady-state conditions. Temperature data must be transmitted to the web dashboard and displayed within 1 second of measurement. Additionally, historical temperature data must be stored and retrievable for at least 24-hour periods within 2 seconds to allow for trend analysis and system evaluation.

Lighting and Daylight Harvesting Requirements:

Lighting must automatically adjust in response to ambient light conditions to improve energy efficiency and maintain consistent illumination levels. The system must measure ambient light intensity and update lighting output within 1 second of a detected environmental change. When users manually adjust lighting through the dashboard, the system must update LED brightness within 300 ms to provide responsive feedback. The lighting system must also support automatic daylight harvesting, adjusting brightness continuously to maintain target illumination levels without user intervention.

Web-Based Monitoring and Control Requirements:

Building managers must be able to monitor system status and control building functions remotely through a centralized dashboard. Environmental data, lighting levels, and access logs must be displayed within 1 second of occurrence to ensure accurate real-time monitoring. The system must maintain at least 99.9% availability during operation to ensure reliable building control. All connected devices must authenticate securely, and the system must reject 100% of unauthorized access attempts to maintain building security.

III. Architecture and/or Principle of Operation

Fig. 1 shows the overall system architecture of the web-based smart building model. The system consists of three primary layers: the web dashboard interface, the backend control system, and the physical house model containing sensing and actuation nodes. These layers communicate through secure network connections to enable centralized monitoring and control of building access, lighting, and environmental conditions.

The web dashboard provides the primary user interface for building managers and security personnel. Implemented using a React frontend, the dashboard allows users to define access policies, manually control lighting, and view historical environmental data. [8] The dashboard communicates with the backend through HTTPS REST requests and WebSocket connections. [9] REST is used for configuration and control commands, while WebSockets provide real-time updates of sensor data and system state.

The backend layer runs on a Raspberry Pi and serves as the central control and coordination system. A FastAPI server handles incoming requests from the dashboard and manages communication with physical devices. [7] Device authentication and secure provisioning are implemented using mutual TLS to ensure that only authorized hardware can connect to the system. Incoming sensor readings and control events are transmitted through MQTT topics, which implement an asynchronous event-driven messaging pipeline. [1] This publish, subscribe architecture enables reliable and low-latency communication between the backend and ESP32 devices. The backend rules engine processes incoming events, enforces access control policies, and generates commands to actuators when required. Environmental and system telemetry data are stored in a TimescaleDB database to support historical analysis and visualization. [6]

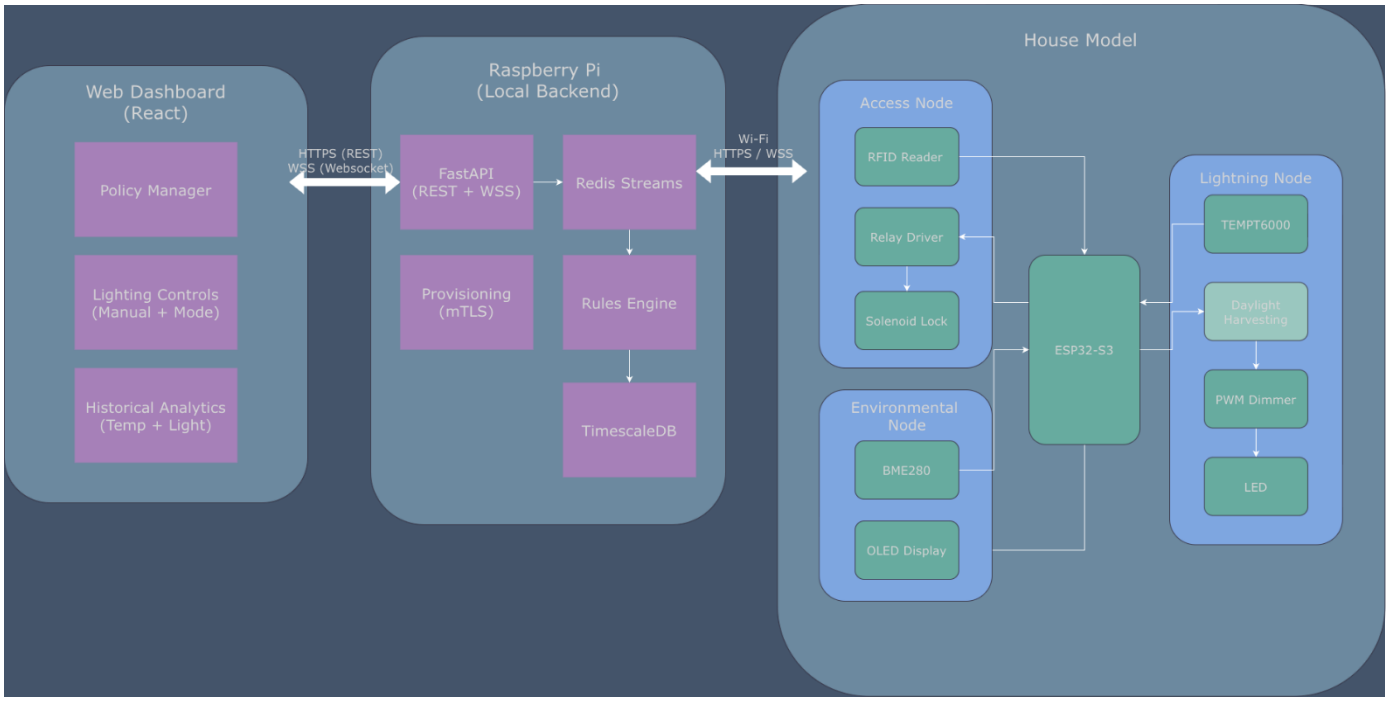
The physical house model represents the controlled building environment and contains three functional subsystems connected to an ESP32-S3 microcontroller. [3] The access control node includes an RFID reader, relay driver, and solenoid lock. When an RFID credential is detected, the ESP32 sends the credential data to the backend for validation. The backend evaluates the request and returns a grant or deny command, which the ESP32 executes by controlling the relay to actuate the door lock.

The environmental monitoring node consists of a BME280 temperature sensor and an OLED display. [2] The ESP32 periodically reads temperature data and transmits it to the backend for storage and visualization. The OLED display provides local feedback on system status and environmental conditions.

The lighting control node implements adaptive lighting using a TEMT6000 ambient light sensor, a daylight harvesting controller, and a PWM dimmer connected to an LED. Ambient light measurements are sent to the backend, where the system determines the appropriate brightness level based on user settings and daylight harvesting algorithms. The ESP32 then adjusts the PWM output to control LED brightness. Lighting can also be controlled manually through the web dashboard.

Communication between the backend and ESP32 occurs over Wi-Fi using secure HTTPS and WebSocket protocols. This enables bidirectional communication, allowing both sensor data transmission and actuator control. The event-driven architecture ensures that sensor readings, access events, and lighting adjustments are processed efficiently and reflected on the dashboard in real time.

Fig. 1. System Architecture Block Diagram from Design Review Presentation.



IV. Design Requirements

The design requirements define the engineering specifications necessary for the system implementation to satisfy the use-case requirements described in Section II. These specifications are derived from the performance, reliability, and functional constraints of centralized building control, including secure access enforcement, closed-loop environmental regulation, and adaptive lighting. The system must ensure low-latency operation, precise environmental control, reliable communication, and secure device authentication to support real-time building automation.

Access Control Subsystem Requirements:

The access control subsystem must authenticate RFID credentials, communicate with the backend server, and actuate the physical door lock. The solenoid lock requires approximately 100 ms for mechanical actuation and operates using a relay driven by the ESP32 microcontroller. To ensure that total unlock latency remains below the use-case requirement, the system must minimize delays in sensing, communication, and processing. The ESP32 must read RFID credentials and transmit them to the backend over a secure wireless connection. The network communication and backend processing time must remain significantly smaller than the mechanical actuation delay. Therefore, the access control subsystem must support secure wireless communication with processing and transmission latency on the order of tens of milliseconds. The relay driver circuit must provide sufficient current to actuate the solenoid reliably while protecting the microcontroller from voltage transients using electrical isolation and flyback protection. Additionally, the system must ensure secure authentication of connected devices. Mutual TLS authentication is required so that only authorized ESP32 devices can communicate with the backend. This prevents unauthorized devices from issuing actuator commands or accessing system data, which is critical for maintaining building security.

HVAC Temperature Regulation Requirements:

The HVAC subsystem must maintain the building temperature near a desired setpoint using closed-loop feedback control. The temperature sensor provides environmental measurements, which are used by the control system to determine the appropriate actuator response. To maintain stable temperature regulation, the sampling rate must be sufficient to detect meaningful changes in environmental conditions while avoiding excessive communication overhead. The temperature sensor selected for the system provides measurement accuracy of approximately ±1 °C. Therefore, the control system must operate with sufficient resolution to maintain temperature regulation within this range. The fan actuator must support variable output levels so that proportional control can be applied rather than simple on-off switching. This is implemented using PWM control signals generated by the ESP32. The control system must also ensure stability and prevent oscillatory behavior. The control loop timing must remain consistent, and sensor readings must be transmitted reliably to the backend. Environmental data must also be stored to allow building managers to analyze trends and evaluate system performance over time.

Lighting and Daylight Harvesting Subsystem Requirements:

The lighting subsystem must adjust the artificial lighting based on both user input and ambient environmental conditions. This requires accurate sensing of ambient light intensity and precise control of LED brightness. The ambient light sensor

produces an analog signal proportional to light intensity, which must be converted to a digital value using the ESP32 analog-to-digital converter. The resolution of the analog-to-digital converter must be sufficient to detect meaningful changes in light intensity. The lighting controller must generate PWM signals to adjust LED brightness smoothly. The PWM resolution and frequency must be high enough to prevent visible flickers while allowing fine brightness adjustment. The lighting system must support both manual control from the dashboard and automatic daylight harvesting. In automatic mode, the system must continuously adjust brightness to maintain consistent illumination levels. This requires periodic sensor sampling, real-time communication with the backend, and responsive actuator control.

Backend and Communication Requirements:

The backend system coordinates communication between the dashboard and physical devices. It must support asynchronous, event-driven processing to ensure responsive system operation. MQTT is used as the primary message broker to distribute system events using a publish–subscribe model, allowing devices and backend services to communicate without blocking. [1] This architecture enables low-latency and scalable communication between multiple system components. The backend must also provide secure communication channels using HTTPS and WebSocket protocols. HTTPS is used for configuration and policy management, while WebSockets enable real-time telemetry updates to the dashboard. Additionally, the backend provides persistent storage of environmental and access data using a time-series database, allowing users to monitor building performance and analyze historical trends.

Microcontroller and Hardware Interface Requirements:

The ESP32-S3 microcontroller serves as the primary interface between sensors, actuators, and the backend system. It must support wireless communication, analog sensor input, digital actuator control, and secure communication protocols. The microcontroller must provide sufficient processing capability to perform sensor sampling, communication, and actuator control simultaneously. It must also support hardware interfaces required by the system, including:

- SPI interface for RFID reader communication
- I2C interface for environmental sensors and display
- ADC interface for ambient light sensing
- PWM interface for actuator control
- GPIO interface for relay control

System-Level Integration Requirements:

The overall system must operate as an integrated platform that combines sensing, control, communication, and user interface functions. The architecture must support real-time monitoring, secure device communication, and reliable actuator control. The event-driven system architecture must ensure sensor data, access control events, and lighting commands are processed efficiently without data loss. It must also support continuous operation without requiring manual intervention.

V. Design Trade Studies

This section describes engineering trade studies that guided the selection of control parameters, communication architecture, actuator resolution, and sensing hardware. These analyses ensure that the system meets requirements for latency, stability, and responsiveness.

A. HVAC Temperature Control Trade Study

The HVAC subsystem regulates temperature using a discrete-time proportional-integral-derivative (PID) controller implemented on the ESP32 microcontroller. The discrete PID control law is given by (1), where $u[k]$ is the actuator output, $e[k]$ is the temperature error, $T_s$is the sampling period, and the K values are the controller gains.

$$u[k] = K_p e[k] + K_i T_{s \sum_{i=0}^{k} e[i]} + K_d \frac{e[k]-e[k-1]}{T_s} \quad (1)$$

The sampling rate directly affects controller stability and derivative accuracy. If the sampling rate is too low, the controller cannot respond quickly to environmental changes. If the sampling rate is too high, communication overhead and noise sensitivity increase.

The thermal dynamics of the system can be approximated as a first-order system shown in (2), where τ is the thermal time constant. Based on enclosure size and airflow characteristics, a conservative estimate is 20 seconds. Then the corresponding bandwidth becomes 0.008 Hz, and our selected sampling rate becomes 1 Hz.

This exceeds the minimum requirement by more than an order of magnitude and ensures stable PID control, accurate derivative estimation, and effective disturbance rejection. Higher sampling rates were considered but rejected because they increased communication overhead without improving control performance.

$$\tau \frac{dT(t)}{dt} + T(t) = Ku(t) \quad (2)$$

B. PWM Resolution and Frequency Trade Study

The HVAC and lighting subsystems use pulse-width modulation (PWM) for actuator control. PWM resolution determines the smallest possible change in actuator output and is defined by (3), where N is the number of bits. Human visual perception can detect brightness changes of approximately 1%. Therefore, N must be greater than or equal to 7. Choosing an 8-bit PWM resolution, it provides resolution = 1/259 = 0.39% that ensures smooth brightness and fan control. PWM frequency must also be sufficiently high to prevent visible flickers. The selected PWM frequency is f_{PWM}= 5 kHz, exceeding the minimum flicker threshold and ensuring stable actuator operation while maintaining efficient processing.

C. Communication Architecture Trade Study

The system requires low-latency communication between ESP32 devices and the backend server. Three communication

architectures were evaluated: polling, Redis Streams, and MQTT.

Polling introduces latency equal to the polling interval, typically 500–1000 ms, which does not meet system responsiveness requirements. Redis Streams provides reliable event delivery but introduces additional processing overhead and complexity. MQTT provides lightweight publish-subscribe communication with typical latency of 10–50 ms. MQTT enables asynchronous communication and allows devices to publish events without waiting for backend processing. Therefore, MQTT was selected as the primary communication protocol because it provides the best balance of low latency, scalability, and implementation simplicity.

D. Access Control Latency Trade Study

Total access control latency consists of RFID read time, network transmission, backend processing, and solenoid actuation is described by (3), where T_{RFID} = 50 ms, $T_{network}$ = 50 ms, $T_{process}$ = 20 ms, $T_{actuation}$ = 100 ms. This makes the total latency to be 220 ms, fulfilling the 500 ms unlock latency requirement.

$$T = T_{RFID} + T_{network} + T_{process} + T_{actuation} \quad (3)$$

E. Lighting Sensor Resolution Trade Study

The ambient light sensor is connected to the ESP32 12-bit analog-to-digital converter, which provides 2^{12} = 4096 measurement levels. For a typical indoor lighting range of 0-1000 lux, this means resolution = 0.24 lux. Human perception threshold is approximately 10 lux, so the selected resolution exceeds the minimum requirement and allows accurate daylight harvesting control.

VI. System Implementation

This section describes the detailed hardware and software implementation of the smart building model. The system integrates three ESP32-based embedded nodes, a Raspberry Pi backend server, and a web dashboard to provide centralized control of access, lighting, and environmental regulation.

The implementation reflects the architecture described in Section III and fulfills the design requirements established in Section IV. The implementation was formulated to address public health, safety, welfare, and environmental considerations. Safety is addressed by fail-safe access control: the door remains locked when the backend or network is unavailable, and mutual TLS ensures only authorized devices can issue commands. Welfare is supported by PID-based temperature regulation and responsive lighting control, providing predictable comfort and visibility. Environmental impact is addressed by daylight harvesting and PWM-based dimming, reducing artificial lighting use when ambient light is sufficient, and by efficient fan control for temperature regulation. These choices keep the solution aligned with the use-case requirements for security, comfort, and energy-aware operation.

A. Access Control

The access control subsystem consists of an ESP32-S3 microcontroller, an RFID-RC522 reader, and a 12 V solenoid door lock controlled through a relay driver circuit. [5] The RFID reader communicates with the ESP32 using the SPI interface. When a card is presented, the reader transmits the card UID to the ESP32. The ESP32 then publishes the UID to the backend server using MQTT over a secure Wi-Fi connection. The backend verifies the UID against the access control database and publishes a grant or deny response.

Upon receiving authorization, the ESP32 activates a relay module connected to the solenoid lock. The relay driver provides electrical isolation and sufficient current to actuate the lock while protecting the microcontroller from inductive voltage spikes. The solenoid requires approximately 100 ms to actuate and remains energized for a fixed unlock interval before automatically returning to the locked state.

The ESP32 firmware for this subsystem was implemented in C++ using the Arduino framework. The firmware includes modules for RFID reading, MQTT communication, and relay control. These functions operate within the main control loop and use non-blocking communication to ensure responsive operation.

B. Environmental Monitoring and HVAC Control

The HVAC subsystem regulates temperature using a BME280 environmental sensor and a PWM controlled fan. The BME280 communicates with the ESP32 using the I2C interface and provides temperature measurements with ±1 °C accuracy. The ESP32 samples the sensor at a rate of 1 Hz and transmits measurements to the backend using MQTT. These measurements are stored in a TimescaleDB database and displayed on the web dashboard. [6]

Temperature regulation is performed using a PID controller implemented on the ESP32. The controller calculates actuator output based on the difference between measured temperature and a user-defined setpoint. The output is converted to a PWM signal that controls fan speed. The ESP32 uses its built-in LEDC PWM hardware to generate a 5 kHz PWM signal. This signal drives a MOSFET-based fan controller, allowing proportional control of airflow and enabling stable temperature regulation. An OLED display connected to the ESP32 provides local feedback by displaying system status and environmental measurements.

C. Lighting Control

The lighting control subsystem implements daylight harvesting using a TEMT6000 ambient light sensor, a PWM dimmer module, and relay-controlled lighting circuits. The TEMT6000 produces an analog voltage proportional to ambient light intensity. This signal is read by the ESP32 analog-to-digital converter and converted to a digital light level measurement. The ESP32 publishes light sensor data to the backend and receives brightness commands through MQTT.

LED brightness is controlled using PWM generated by the ESP32. [3] The PWM signal drives a MOSFET dimmer module, which regulates LED power efficiently. The selected PWM frequency of 5 kHz ensures flicker-free operation and smooth brightness transitions.

The lighting subsystem also includes a 4-channel relay module that allows switching of higher-power loads. These relays are controlled through ESP32 GPIO pins and enable digital control of lighting and auxiliary devices. In automatic mode, the ESP32 should adjust brightness based on ambient light measurements to maintain consistent illumination levels.

D. Backend Server Implementation

The backend server runs on a Raspberry Pi and is implemented using the FastAPI Python framework. [7] The backend serves as the central coordination system and manages communication between the web dashboard and ESP32 devices. The backend uses an MQTT broker to implement publish–subscribe communication. ESP32 devices publish sensor data and access requests to MQTT topics, and the backend publishes control commands and authorization responses. Additionally, it includes a rules engine that processes incoming events, enforces access control policies, and generates actuator commands.

TimescaleDB is used to store environmental and access control data. TimescaleDB extends PostgreSQL to support efficient storage and querying of time-series data. This allows users to view historical trends and analyze system performance. The backend also provides REST API endpoints for dashboard configuration and WebSocket connections for real-time telemetry updates. Secure communication is implemented using TLS encryption and mutual authentication to ensure that only authorized devices can connect to the system.

E. Web Dashboard Implementation

The web dashboard provides the primary user interface for system monitoring and control. The dashboard was implemented using the React framework and communicates with the backend using HTTPS and WebSocket protocols. [8] The dashboard displays real-time environmental data, lighting levels, and access logs. Users can control lighting brightness, modify temperature setpoints, and manage access permissions. WebSocket communication allows the dashboard to receive real-time updates without requiring page refreshes. This enables responsive system monitoring and control.

F. System Integration

The three layers, physical house model, backend server, and web dashboard, operate as a single closed-loop system. End-to-end flow begins at the sensors: each ESP32-S3 reads local sensors (BME280, light sensor, RFID reader) and publishes measurements and events to the backend over Wi-Fi. The backend receives these messages via MQTT, applies the rules engine (e.g., access policy checks, PID or temperature-equalization logic, daylight-harvesting setpoints), and updates state in TimescaleDB. Control commands and authorization responses are published back over MQTT to the appropriate ESP32 devices, which drive actuators (relay for the solenoid lock, PWM for fans and LED dimmers). The dashboard connects to the backend over HTTPS and WebSockets; REST calls are used for configuration and manual control, while WebSockets push real-time telemetry so the UI reflects sensor data and actuator state within the required latency (e.g., under 1 s for environmental updates). MQTT provides low-latency, asynchronous device-to-backend messaging, and WebSockets provide low-latency backend-to-dashboard updates, so the full path from sensor to display and from user command to actuator is consistent with the system's performance targets. If the backend or network is unavailable, ESP32 nodes continue to run local control where implemented (e.g., PID or fan logic) but cannot receive new policies or dashboard commands; access control fails secure (e.g., door remains locked) when backend authorization cannot be obtained.

VII. Test, Verification and Validation

This section describes the testing methods we will use to verify that the implementation meets the design requirements in Section IV and to validate that it satisfies the use case of centralized building management. Tests will be conducted on individual subsystems and on the fully integrated system. Pass/fail criteria are derived from the design and use-case requirements.

A. Access Control Latency Verification

We will verify unlock latency and authorization correctness. Latency will be measured from RFID detection to solenoid actuation using firmware timestamps over a set of trials (e.g., 100 swipes). Pass: average latency < 500 ms and maximum latency < 500 ms; all unauthorized credentials rejected. Fail: any trial exceeds 500 ms or any unauthorized credential is accepted. We will also confirm that revoked credentials are rejected within 100 ms of a policy change and that 100% of access attempts are logged with timestamps.

B. HVAC Monitoring and Control Verification

We will verify sampling rate and temperature regulation. Sensor timestamps will be used to confirm a 1 Hz sampling rate. Pass: sampling at 1 Hz; temperature regulated within ±0.5 °C of setpoint under steady-state conditions; environmental data logged and displayed on the dashboard within 1 s. Fail: sampling below 1 Hz, steady-state error outside ±0.5 °C, or data not visible within 1 s. Historical data retrievability (24-hour window within 2 s) will be checked separately.

C. Lighting Control Verification

We will verify manual control and daylight harvesting. We will send brightness commands from the dashboard and measure response time (e.g., via firmware or oscilloscope). Pass: LED brightness changes within 300 ms of a manual command; ambient light changes produce automatic brightness adjustments; PWM frequency at 5 kHz (verified with oscilloscope). Fail: response > 300 ms, no automatic adjustment to ambient changes, or incorrect PWM frequency.

D. Communication and Integration Validation

We will run the system continuously (e.g., 24 hours) and verify end-to-end data flow. Pass: all sensor data transmitted, stored, and displayed without loss; dashboard updates within 1 s of measurement; access control, lighting, and environmental monitoring operate simultaneously without failures. Fail: any data loss, updates consistently > 1 s, or subsystem failures during combined operation.

VIII. Project Management

A. Schedule

The project schedule spans 12 weeks and is organized into five phases. Phase One (Weeks 1–3) covers system architecture, backend API and MQTT broker setup, and the TimescaleDB schema. Phase Two (Weeks 4–5) and Phase Three (Weeks 6–7) cover firmware development for access control, environmental/HVAC, and lighting, frontend dashboard development, and hardware wiring and bring-up. Phase Four (Weeks 8–10) focuses on firmware–backend integration, dashboard–backend integration, and full system integration. Phase Five (Weeks 11–12) is reserved for latency and requirements verification and for debugging, hardening, and final demo preparation. A detailed Gantt chart showing task ownership (Cindy, Mario, or both), weekly and daily breakdown (Monday–Friday), and phase boundaries is provided as Fig. 2. Key milestones are the completion of each phase (Weeks 3, 5, 7, 10, and 12) and the transition from integration to verification at the start of Week 11.

B. Team Member Responsibilities

Mario Belmonte (MB): Primary responsibilities: backend and web stack (FastAPI services, MQTT topics, TimescaleDB schema, React dashboard), Raspberry Pi setup, CI pipeline and test coverage, and access-control/authorization logic. He also owns security configuration (TLS certificates, device provisioning) and overall integration of the backend with the three ESP32 subsystems. Secondary responsibilities: assisting with firmware APIs and debugging ESP32–backend communication, helping design and tune equalization logic across rooms, and contributing to documentation and final report.

Cindy Chen (CC): Primary responsibilities: embedded firmware and hardware integration, including ESP32 bring-up, wiring of RFID, BME280, fans, light sensors, dimmers, and relays, as well as PID temperature control and lighting control logic. She also leads system architecture diagrams, requirements mapping, and test/validation planning (latency measurements, multi-room behavior, and safety checks). Secondary responsibilities: contributing to dashboard UX and control flows, maintaining the Gantt chart and risk tracking, and co-authoring user-facing documentation.

Both team members share responsibility for refining requirements, design decisions, integration testing, and preparing presentations and demos.

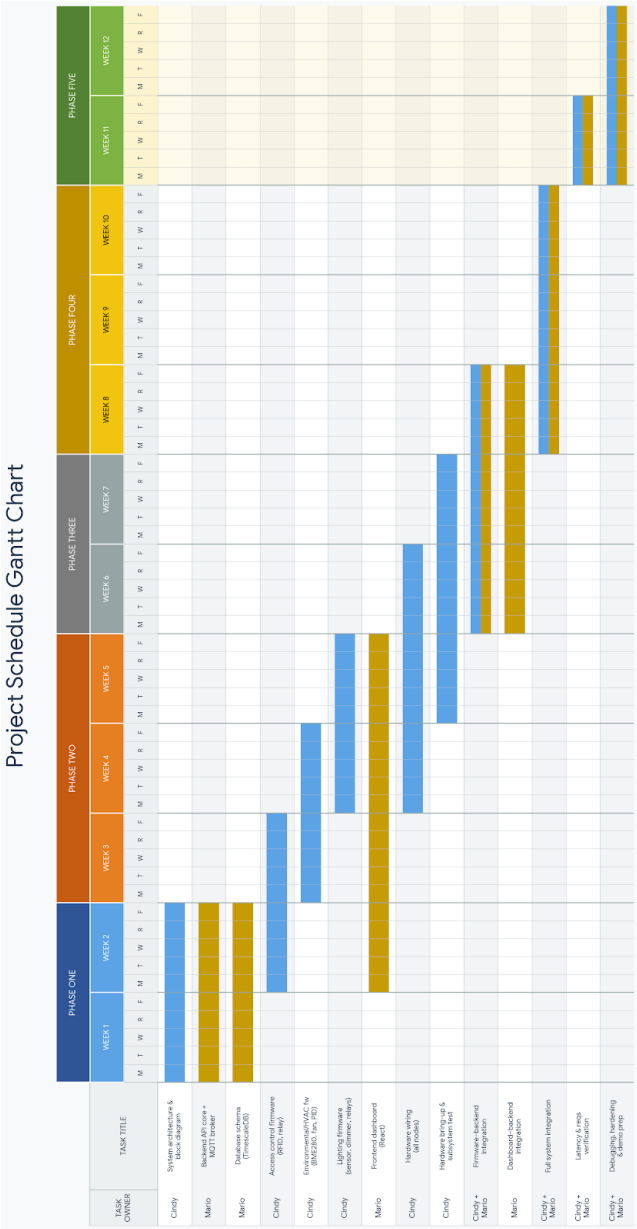


Fig. 2. Schedule example with milestones and team responsibilities

C. Bill of Materials and Budget

The complete bill of materials, including description, model number, manufacturer, quantity, unit cost, and line total for each component, is given in Table I at the end of this report. The total cost of purchased parts for the smart building model is $225.68.

D. TechSpark Use Plan

We do not plan on meeting up at TechSpark for this project.

E. Risk Mitigation Plans

Critical risks and planned mitigations are as follows:

Network and backend availability:

ESP32 nodes depend on Wi-Fi and the Raspberry Pi backend for access control and coordinated HVAC/lighting. Link drops or Pi outages can delay or block authorization and commands.

Mitigation: Use stable Wi-Fi (dedicated AP if needed), keep the Pi on wired Ethernet, and implement fail-safe behavior in firmware (e.g., door remains locked if the backend is unreachable; local fan/PID can continue from last valid setpoint or default to a safe mode).

Multi-room temperature control stability:

Balancing temperature across three rooms with PID-driven fans is a new control problem for the team; poor tuning or sensor placement could cause oscillation or slow convergence. Mitigation: Start with conservative PID gains and 1 Hz sampling as in the trade study; test in the physical model and tune in stages; log temperature and fan commands to the backend for offline analysis and gain adjustment.

End-to-end latency:

Meeting <500 ms for access control and <1 s for environmental updates depends on MQTT, backend processing, and WebSocket updates. Unknown network load or backend delays could exceed targets. Mitigation: Measure latency in integration tests (RFID swipe to lock actuation, sensor read to dashboard update); add simple timing logs or metrics in the backend and firmware; optimize or offload heavy work if measurements approach the limits.

Hardware and integration reliability:

Multiple ESP32s, sensors, relays, and power supplies increase the chance of wiring errors, marginal connections, or one faulty part stalling integration. Mitigation: Test each subsystem (access, one room's BME280, fan, lighting) individually before full integration; keep a spare ESP32-S3 and critical parts (e.g., relay, MOSFETs) for swap-out; document pinout and power rails in a single reference so debugging is straightforward.

IX. Related Work

There's much to learn from commercial examples and closely related student projects. The prior team project SmartWatt [10] targets energy and cost savings through machine-learning-based forecasting and scheduling of appliances around solar and grid pricing. In contrast, our Smart Home Model treats the building as critical infrastructure: access, environmental control, and lighting are managed entirely through a centralized web-based system, with emphasis on security, policy enforcement, and real-time responsiveness rather than energy optimization alone.

From SmartWatt's implementation and reports we took three lessons, adapted to our current design. First, latency: their stack (FastAPI, MQTT) is suitable for backend APIs where end-to-end delay is secondary; they report delays on the order of seconds. We target <500 ms for access control and <1 s for environmental and lighting updates. To support that we use WebSockets for live dashboard updates and MQTT for device-to-backend messaging, with TLS for secure links, so the path from sensor or RFID event to dashboard and actuator stays within our targets. Second, sensor and data quality: SmartWatt used high-precision parts (e.g., INA226) and documented calibration (e.g., ~0.5% power error). We use specified-accuracy sensors (e.g., BME280 for temperature) and TimescaleDB for time-series storage so that historical analytics and tuning are based on quantifiable, logged data. Third, asynchronous, event-driven design: SmartWatt experienced UI freezes when heavy work ran synchronously (e.g., "Optimize Schedule" blocking the response). They addressed this with FastAPI background tasks. We avoid blocking the UI and backend by using an event-driven message broker (MQTT as primary) so devices publish events and the backend processes them asynchronously; the dashboard receives updates over WebSockets and does not wait on long-running requests. In summary, our architecture prioritizes secure, low-latency, policy-driven control with MQTT and WebSockets for responsiveness and TimescaleDB for persistence, differing from SmartWatt's energy-scheduling focus while building on similar infrastructure choices (FastAPI, MQTT, time-series data).

Commercial smart home and building automation products (e.g., smart locks, thermostats, lighting systems) typically offer local or cloud control but use proprietary ecosystems and distributed logic, which limits integration and transparency. Our project instead explores a single, web-first control plane with explicit latency and availability targets and full observability of access, environmental, and lighting state.

X. Summary

This project presents a web-first smart building model that unifies RFID access control, multi-room HVAC temperature regulation via PID-controlled fans, and adaptive lighting with daylight harvesting. All sensing and actuation are coordinated through a Raspberry Pi backend and a React dashboard, so building managers and security staff get a single place to monitor and control access, room temperatures, and lighting in near real time. For stakeholders, the impact is centralized visibility and policy enforcement: access rules and revocations apply immediately, environmental and lighting data are logged for analysis, and the system is designed to meet explicit latency targets of <500 ms for door unlock, <1 s for environmental and lighting updates), supporting responsive, audit-friendly building operation.

Implementation challenges include integrating and tuning the three-room thermal model ensuring PID or equalization logic keeps temperatures stable and balanced across rooms without oscillation and maintaining end-to-end latency under load. Hardware integration (multiple ESP32 nodes, BME280s, fans, relays, and power) increases the chance of wiring and connectivity issues, so subsystem-level testing and clear documentation will be important. Validating access-control latency and dashboard update times in a live network, and hardening fail-safe behavior when the backend or Wi-Fi is unavailable, will also be critical.

Meeting the use-case and design requirements will depend on reliable MQTT and WebSocket connectivity, accurate sensor readings and calibration, and backend availability. Risks include network variability affecting the <500 ms access target, and multi-room control stability depending on sensor placement and tuning. Addressing these through measured latency tests,

staged PID tuning, and defined offline/fail-safe behavior will determine how well the built system matches the intended use case of secure, responsive, centralized building management.

Glossary of Acronyms

ADC – Analog-to-Digital Converter
API – Application Programming Interface
BOM – Bill of Materials
ESP32 – Espressif 32-bit Wi-Fi/Bluetooth microcontroller
GPIO – General-Purpose Input/Output
HTTPS – Hypertext Transfer Protocol Secure
I²C – Inter-Integrated Circuit (two-wire serial bus)
LED – Light-Emitting Diode
LEDC – LED Controller (ESP32 PWM hardware peripheral)
MQTT – Message Queuing Telemetry Transport
PID – Proportional–Integral–Derivative (control algorithm)
PWM – Pulse-Width Modulation
REST – Representational State Transfer
RFID – Radio-Frequency Identification
RPi – Raspberry Pi
SPI – Serial Peripheral Interface
TLS – Transport Layer Security
UI – User Interface
WS / WSS – WebSocket / WebSocket Secure

References

[1] OASIS, "MQTT Version 3.1.1 Plus Errata 01," OASIS Standard, Dec. 2015. [Online]. Available: https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/
[2] Bosch Sensortec, "BME280 Combined Humidity and Pressure Sensor," Data Sheet, Rev. 1.6, Oct. 2018.
[3] Espressif Systems, "ESP32-S3 Series: 2.4 GHz Wi-Fi and Bluetooth 5 (LE) System-on-Chip," Datasheet, v1.6, 2023.
[4] Rohm Semiconductor, "BH1750FVI Ambient Light Sensor IC," Technical Note, Rev. B, 2013.
[5] NXP Semiconductors, "MFRC522 Standard Performance MIFARE and NTAG Frontend," Data Sheet, Rev. 3.9, 2016.
[6] Timescale, Inc., "TimescaleDB: A Time-Series Database for PostgreSQL," White Paper, 2017. [Online]. Available: https://www.timescale.com/
[7] S. Raposo and contributors, "FastAPI Documentation," [Online]. Available: https://fastapi.tiangolo.com/ (accessed Feb. 21, 2026).
[8] Meta Platforms, Inc., "React: A JavaScript Library for Building User Interfaces," [Online]. Available: https://react.dev/ (accessed Feb. 21, 2026).
[9] I. Fette and A. Melnikov, "The WebSocket Protocol," RFC 6455, Dec. 2011.
[10] Team SmartWatt, "SmartWatt: Intelligent Residential Energy Management Using Predictive Scheduling," Carnegie Mellon Univ., ECE Capstone Project, Spring 2025, Final Report and Project Website, [Online]. Available: https://course.ece.cmu.edu/~ece500/projects/s25-teamd5/ (accessed Feb. 21, 2026).

Table I. Bill of Materials

Description	Model #	Manufacturer	Quantity	Cost @	Total
30×30×10 mm 5 V brushless fans	30x30x10mm 5V	Generic	3	$6.99	$20.97
BME280 temperature/humidity/pressure sensors (HiLetgo 3.3 V GY-BME280-3.3)	GY-BME280-3.3	HiLetgo	3	$8.99	$26.97
12 V mini solenoid door lock	12V mini solenoid	Generic	1	$16.99	$16.99
5 V 1-channel relay module with optocoupler	5V 1-CH Relay	Generic	1	$8.49	$8.49
RC522 / MFRC522 RFID reader kit (SPI, 13.56 MHz, with cards)	MFRC522	D-FLIFE	1	$10.99	$10.99
Low-voltage LED PWM dimmers (12 V, knob style) for room lights	12V PWM Dimmer	SUPERNIGHT	3	$9.99	$29.97
Ambient light sensors (BH1750 I^2C modules)	BH1750	NOYITO	3	$6.99	$20.97
USB wired keyboard (for Raspberry Pi 5)	USB Keyboard	Generic	1	$8.99	$8.99
ESP32-S3 DevKit-style board	ESP32-S3 DevKit	Generic	1	$30.99	$30.99
Pack of N-channel logic-level MOSFETs (IRLZ44N or similar, TO-220)	IRLZ44N	Bestol	2	$9.99	$19.98
Pack of flyback diodes (1N4007, 100 pcs)	1N4007	EEEEE	1	$5.99	$5.99
12 V 2 A regulated power supply (with interchangeable DC tips)	12V 2A PSU	Generic	1	$11.99	$11.99
5 V 3 A regulated power supply	5V 3A PSU	Generic	1	$8.39	$8.39
LM2596-based buck converter module (12 V → 5 V, 3 A)	LM2596	Generic	1	$4.00	$4.00
Subtotal					$225.68

All components can be bought through Amazon